

Capítulo 1

Marco de Trabajo (Framework)

1.1. Introducción

La instrumentación científica en laboratorios de física experimental enfrenta frecuentemente el desafío de la falta de reproducibilidad y el empaquetamiento propietario...

1.2. Marco Teórico

1.2.1. Paradigma de Co-diseño Hardware/Software

1.2.2. Filosofía y Ecosistema PYNQ

1.2.3. Filosofía y Gestión DevOps con Hog

1.3. Metodología

1.3.1. Estrategia de Repositorios Duales (HW/SW)

1.3.2. Flujo de Trabajo CI/CD y Transducción AXI/DMA

1.3.3. Justificación: Instrumentación Libre, Modular y Reusable

1.4. Resultados y Validación

1.4.1. Plataforma de Adquisición: `hw-xadc-dma-overlays`

Para validar el marco de trabajo propuesto y establecer una plataforma preliminar de pruebas para sistemas de adquisición de datos (DAQ), se desarrolló el repositorio de firmware `hw-xadc-dma-overlays`, orientado a la tarjeta SoC-FPGA PYNQ-Z2 (dispositivo Zynq-7000 xc7z020c1g400-1). Este proyecto implementa una plataforma de adquisición en la Lógica Programable (PL) capaz de digitalizar señales analógicas a alta velocidad y transferirlas hacia la memoria DDR del procesador (PS) sin intervención de la CPU.

Arquitectura de Adquisición y Flujo AXI4-Stream

La arquitectura de adquisición se diseñó utilizando Vivado 2024.2.2 bajo la metodología de IP Integrator (Figura 1.1). El flujo de señal se articula mediante los siguientes bloques principales:

1. **Digitalizador XADC** (`xadc_wiz_0`): Configurado en modo canal único sobre la entrada auxiliar diferencial `VAUXP1/VAUXN1`, operando a una frecuencia de muestreo de 1 MSPS (1000 kSPS). La interfaz de salida fue establecida en formato AXI4-Stream de 16 bits (`M_AXIS`).
2. **Generador de TLAST** (`tlast_generator`): Módulo VHDL personalizado que actúa como empaquetador de flujo, asertando la señal

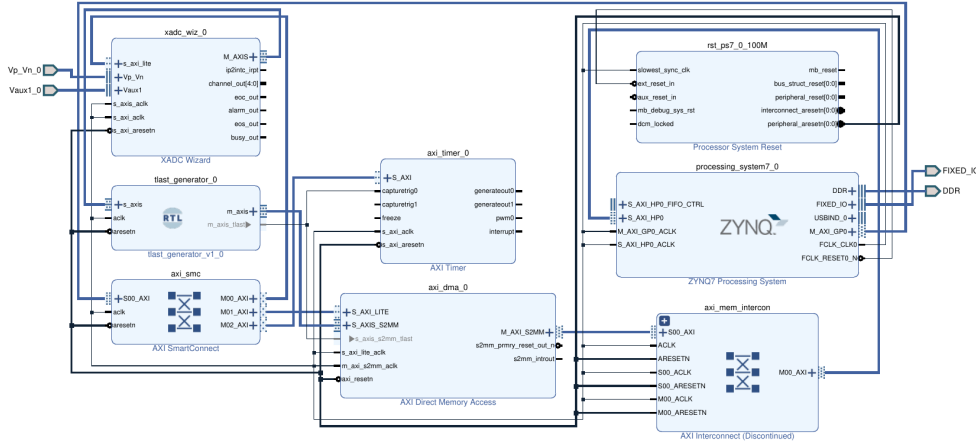


Figura 1.1: Diagrama de bloques general en Vivado IP Integrator para la plataforma de adquisición hw-xadc-dma-overlays.

TLAST combinatorialmente cada $N = 16\,384$ muestras (PACKET_SIZE) para delimitar las ráfagas de memoria requeridas por el motor DMA.

3. **Acceso Directo a Memoria (axi_dma_0):** Motor DMA operando en modo *Simple DMA* directo (S2MM) con un tamaño de ráfaga de 128 transferencias. Los datos son transmitidos a través del interconector de datos (axi_mem_intercon), adaptando el flujo de 32 bits del DMA hacia el bus de alto rendimiento AXI-HP0 de 64 bits del Zynq PS. Esta transferencia directa a memoria DDR (0x00000000 – 0x1FFFFFFF) alcanza un *throughput* sostenido de hardware de 1.75 MB/s, liberando por completo a la CPU de tareas de procesamiento de muestra por muestra.
4. **Temporizador AXI y Medición de Latencia (axi_timer_0):** Bloque de apoyo cuya entrada de captura (capturetrig0) se conecta a la señal TLAST, permitiendo registrar con precisión de 10 ns (100 MHz) la finalización de paquetes desde la PL.
5. **Infraestructura de Interconexión y Reloj:** El bloque processing_system7_0 genera un reloj de sistema de 100 MHz (FCLK_CLK0) y un reseteo sincronizado mediante rst_ps7_0_100M. La distribución de comandos de control desde el PS se gestiona mediante el conmutador AXI Smart-Connect (axi_smc) sobre el bus M.AXI-GP0, mapeando las direcciones

de registro del DMA (0x40400000), del temporizador (0x42800000) y del XADC (0x43C00000).

Gestión de Firmware con CERN Hog y Despliegue CI/CD

La construcción y control de versiones del proyecto se gestionaron mediante CERN Hog (Top/pynq_z2/), utilizando archivos de listas (.src) para desacoplar el código VHDL y el diagrama de bloques (xadc.bd) de los archivos binarios propietarios de Vivado.

El *wrapper* de nivel superior (xadc_wrapper.vhd) incorpora genéricos de versión (GLOBAL_VER, GLOBAL_SHA), embebiendo dinámicamente el *hash* de Git en los registros de hardware durante la síntesis. Un script TCL de gancho (post-bitstream.tcl) extrae automáticamente los archivos binarios compilados (pynq_z2.bit y pynq_z2.hwh) a la carpeta bin/. Finalmente, un flujo de integración continua en GitHub Actions (release.yml) empaqueta y publica automáticamente estos artefactos binarios en los lanzamientos del repositorio al detectar etiquetas de versión (v*).

1.4.2. Interfaz e Instrumentación: sw-pynq-oscilloscope

Para abstraer la complejidad del hardware en la Lógica Programable (PL) y ofrecer una interfaz interactiva de alto nivel, se desarrolló la librería en Python pynq-oscilloscope (sw-pynq-oscilloscope), disponible en PyPI (pip install pynq-oscilloscope). Esta capa de software permite controlar el hardware de adquisición, automatizar estímulos analógicos mediante hardware en el lazo (HIL) y visualizar señales en tiempo real en entornos Jupyter Notebook.

Carga Autónoma de Hardware y Controladores DMA

- **Carga Autónoma (HardwareLoader):** Inspecciona hardware.json (v1.0.2), detecta el modelo de la tarjeta objetivo (\$BOARD), consulta la API de GitHub Releases, descarga los archivos binarios pynq_z2.bit y pynq_z2.hwh a un directorio de caché local (~/.cache/pynq-oscilloscope/v1.0.2/) y carga la firma en la FPGA.
- **Controlador DMA (StreamingXADC):** Realiza una preasignación de memoria contigua (CMA) fuera del bucle de captura mediante pynq.allocate(shape=(16384, dtype='u2')). Al invocar capture(), desencadena la transferencia DMA S2MM, espera la aserción de TLAST en la muestra 16384, realiza el alineamiento de bits (raw >> 4) y escala la señal al rango físico (0.0 V a 3.3 V).

Estímulo HIL y Cuadro de Mando Interactivo

Para validar la respuesta dinámica del framework sin instrumental externo costoso, se integró el instrumento Digilent Analog Discovery 3 (AD3) mediante la librería `pydwf`:

- **Gestión de Entorno (`EnvironmentManager`):** Instala automáticamente los paquetes Debian de Digilent Adept y WaveForms SDK para arquitecturas ARM (32/64 bits) y gestiona los permisos de bus USB (`/dev/bus/usb/`).
- **Estímulo HIL No Bloqueante (`AD3SignalGenerator`):** Ejecuta la generación de señales (seno, cuadro, triángulo) en un hilo secundario (*daemon thread*), permitiendo modificar parámetros en tiempo real sin congelar el kernel de Jupyter.
- **Panel Interactivo (`OscilloscopeDashboard`):** Construido con `ipywidgets` y Plotly `FigureWidget` (Figura 1.2). Implementa enlaces de cliente de cero latencia (`widgets.jslink()`), disparo por software por flanco de subida (`np.where`) y renderizado atómico (`fig.batch_update()`).

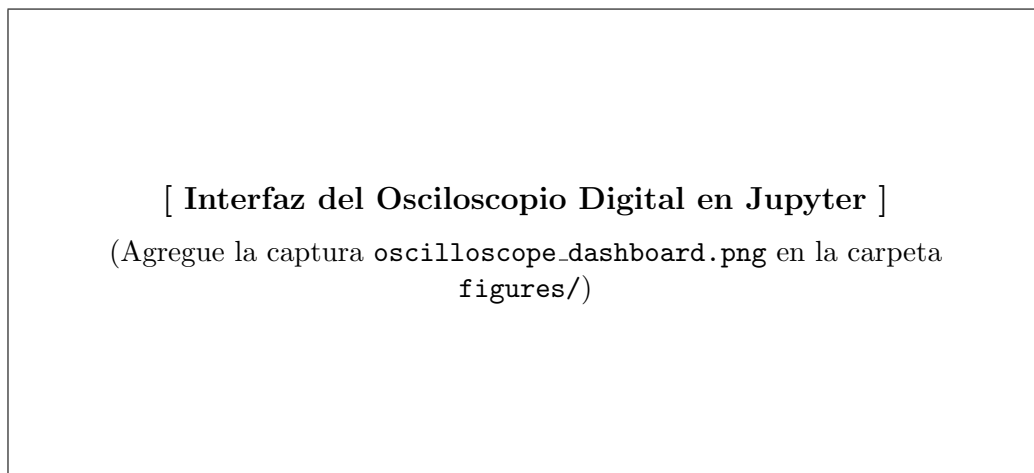


Figura 1.2: Cuadro de mando interactivo del osciloscopio digital en tiempo real desplegado dentro de un entorno Jupyter Notebook.

Validación del Framework mediante Caracterización HIL

La suite de caracterización en lazo cerrado (`04_automated_performance_benchmark.ipynb`) sirvió para verificar experimentalmente que el flujo de co-diseño propuesto preserva la fidelidad de la señal y alcanza un alto rendimiento de software:

- **Respuesta y Calidad de Señal:** Se midió un ancho de banda analógico a -3 dB de 150.0 kHz, una relación señal-ruido de $\text{SNR} = 45.96$ dB ($\text{ENOB} = 7.34$ bits para entrada senoidal de 50 kHz) y un error medio de offset DC menor a 9.69 mV (precisión estática $> 99.7\%$).
- **Rendimiento de Interfaz:** El panel interactivo alcanzó una tasa de refresco fluida de 53.3 cuadros/s (FPS) con un *throughput* sostenido de 1.75 MB/s, demostrando que la capa PYNQ procesa eficazmente el flujo masivo de datos proveniente de la PL.

1.4.3. Proyección a Módulos de Localización Espacial

La implementación y caracterización experimental de los repositorios `hw-xadc-dma-overlays` (Sección 1.4.1) y `sw-pynq-oscilloscope` (Sección 1.4.2) constituyen la prueba de concepto que valida la solidez del marco de trabajo (*framework*) co-diseñado. Los resultados confirman que la combinación de los entornos CERN Hog y AMD PYNQ permite construir instrumentación científica abierta de alto rendimiento, logrando transferencias deterministas por DMA a 1.75 MB/s y tasas de refresco interactivas de 53.3 cuadros/s sin generar cuellos de botella en la CPU ni en la interfaz gráfica.

Un aspecto fundamental demostrado en este capítulo es el desacoplamiento arquitectónico entre la capa de hardware en la Lógica Programable (PL) y la capa de abstracción en software (PS):

- **Infraestructura Consolidada e Invariante:** El flujo de transporte de datos impulsado por eventos (AXI4-Stream), el empaquetado de muestras (`tlast_generator`), el motor AXI DMA sobre el bus de 64 bits AXI-HP0, la asignación de memoria contigua en Python (`StreamingXADC`) y la carga autónoma de artefactos (`HardwareLoader`) se mantendrán como la columna vertebral invariable del sistema.
- **Escalabilidad hacia la Localización de Fuentes de Ondas:** En los capítulos siguientes, el módulo de prueba de canal único de la XADC se sustituirá por arreglos geométricos de sensores multicanal e integraciones con núcleos IP de procesamiento digital de señales (DSP) en la PL (Capítulo 2 y Capítulo 3). Debido a que la tubería (*pipeline*) AXI DMA y la capa PYNQ son agnósticas al tipo de dato transmitido, la adición de algoritmos de estimación de ángulo de llegada (AoA) o diferencia de tiempo de llegada (TDOA) en hardware no requerirá reestructurar la capa de distribución en software ni la interfaz de usuario.

De este modo, el marco de trabajo no solo resuelve el problema de la falta de reproducibilidad y versionado en firmware FPGA, sino que garantiza que

la plataforma completa para la localización espacial de fuentes de ondas se construya sobre una base metodológica probada, modular y completamente abierta.

1.5. Conclusiones del Capítulo

1. Se estableció una metodología de co-diseño hardware/software para instrumentación científica abierta basada en los entornos CERN Hog y AMD PYNQ, resolviendo la falta de trazabilidad y el desacoplamiento tradicional entre el desarrollo de firmware FPGA y las aplicaciones de usuario.
2. La integración de CERN Hog en el repositorio de hardware (`hw-xadc-dma-overlays`) permitió automatizar la construcción de proyectos en Vivado mediante scripts TCL y pipelines de CI/CD en GitHub Actions, garantizando que cada artefacto ejecutable (`.bit` y `.hwh`) esté vinculado de forma inequívoca al *hash* de un commit de Git.
3. La capa de software (`sw-pynq-oscilloscope`), distribuida mediante PyPI, demostró la viabilidad de empaquetar plataformas SoC-FPGA como librerías estándar en Python. El módulo `HardwareLoader` elimina la necesidad de configuración manual de archivos de firmware al descargar autónomamente los mapas de bits requeridos desde los lanzamientos del repositorio de hardware.
4. El empaquetador VHDL diseñado (`tlast_generator`) resolvió la limitación nativa de delimitación de paquetes del núcleo XADC, permitiendo transferencias continuas por AXI DMA hacia la memoria RAM compartida a 1.75 MB/s con una latencia de hardware medible a 100 MHz mediante el temporizador AXI.
5. La suite de caracterización en lazo cerrado validó la fidelidad del flujo de adquisición, alcanzando un ancho de banda analógico de 150.0 kHz, una relación señal-ruido de 45.96 dB ($\text{ENOB} = 7.34$ bits), una precisión estática superior al 99.7 % y un despliegue gráfico fluido a 53.3 FPS. Esta infraestructura validada servirá como la base operativa para la implementación del localizador espacial de fuentes de ondas en los capítulos posteriores.